

NAML PROJECT

Reinforcement Learning in Discrete and Continuous Environments

Technical Project Report

Flag collection, cliff walking, and differential-drive navigation

Mathematical foundations, implementation, and experimental analysis

September 2026

Contents

1	Introduction and mathematical foundations	2
1.1	Project objectives and evidence	2
1.2	MDPs, observations, and return	2
1.3	Why the Bellman operator is a contraction	3
2	Discrete environments	3
2.1	Shared tabular algorithms	3
2.1.1	Q-learning: an off-policy target	3
2.1.2	SARSA: learning from the behavior policy	4
2.1.3	What these guarantees mean for the project	4
2.1.4	Recorded discrete experiments	4
2.2	Flag collection environment	5
2.2.1	Environment specification	5
2.2.2	Training and implementation	6
2.2.3	Training results and greedy evaluation	6
2.3	Cliff walking environment	11
2.3.1	Environment specification	11
2.3.2	Training configuration	11
2.3.3	Why this task distinguishes SARSA from Q-learning	11
2.3.4	Training results and the exploration tradeoff	12
3	Continuous differential-drive navigation	13
3.1	Environment, dynamics, and observations	13
3.1.1	Obstacle generation	14
3.1.2	Active reward function	14
3.2	Shared deep-learning implementation	15
3.3	DDPG: deterministic actor and a single critic	16
3.4	TD3: reducing actor–critic estimation error	16
3.5	SAC: stochastic control and entropy regularization	17
3.5.1	Soft policy iteration and its limits	17
3.5.2	Implemented actor, critics, and temperature	18
3.6	Training configuration and comparison protocol	18
3.7	Training results and interpretation	20
3.7.1	DDPG: strong improvement with temporary regressions	20
3.7.2	TD3: limited improvement and persistent cutoffs	23
3.7.3	SAC: rapid early improvement and adaptive temperature	26
3.7.4	Comparison of the selected training runs	30
3.8	Evaluation evidence and historical shared-map benchmark	31
3.8.1	Historical three-agent benchmark	31
3.8.2	Current DDPG evaluation	31
4	Discussion and limitations	32
4.1	Representation, exploration, and theoretical guarantees	32
4.2	Time limits and partial observations	32
4.3	Experimental evidence and reproducibility	32
5	Conclusions	33
	References	35

1 Introduction and mathematical foundations

1.1 Project objectives and evidence

This project studies reinforcement learning (RL) in three implemented environments: a grid in which an agent collects flags, a cliff-walking grid in which exploratory mistakes are expensive, and a continuous navigation simulator for a differential-drive robot with LiDAR. The two grids use tabular Q-learning and SARSA. The robot uses Deep Deterministic Policy Gradient (DDPG), Twin Delayed DDPG (TD3), and Soft Actor-Critic (SAC).

The environments address complementary questions. Flag collection requires the agent to distinguish otherwise identical positions according to which objectives remain. Cliff walking exposes the difference between learning about a greedy target policy and learning about the policy actually used for exploration. Continuous navigation introduces continuous actions, neural approximation, limited observations, and obstacle avoidance. Across these tasks, the report connects update equations and theoretical guarantees to the actual implementation and available experimental evidence.

The implementation descriptions refer to the corrected environment and agent source files and their current training entry points. The continuous training analysis uses completed DDPG, TD3 and SAC runs from 10 September 2026; the discrete analysis uses completed Q-learning and SARSA runs in both grids from 13 September 2026. Figures and numerical claims are tied to saved configurations and episode metrics. The four grid checkpoints also have saved greedy evaluations. The historical three-agent fixed-map robot benchmark is retained separately and does not evaluate the newly trained continuous policies. Current DDPG evaluation logs provide additional limited evidence. These are individual-run case studies; the planned comparison across independent training seeds has not been performed. The separate moving-obstacle prototype and introductory Gymnasium notebooks are outside the three-environment comparison.

1.2 MDPs, observations, and return

An idealized fully observed task is described by a Markov decision process

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle, \quad 0 \leq \gamma < 1.$$

Here $s_t \in \mathcal{S}$ is the state, $a_t \in \mathcal{A}$ is the action, and P describes the next-state distribution. We use the convention that r_{t+1} is the reward returned after executing a_t . The reward may depend on the entire transition, $R(s_t, a_t, s_{t+1})$. For a policy π , define

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \tag{1}$$

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s], \tag{2}$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]. \tag{3}$$

An absorbing terminal state has zero subsequent value. The optimal action-value function satisfies the Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \sup_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right]. \tag{4}$$

For finite action sets the supremum is a maximum. Attainment and suitable regularity are required when discussing an optimal action in a continuous action set.

The agent need not receive the complete simulator state. We write o_t for an observation and z_t for the normalized observation supplied to a neural network. In the robot task, the obstacle

layout and absolute robot pose are not supplied directly. Consequently, an observation is not guaranteed to be a sufficient Markov state. The theoretical results below concern their stated MDP assumptions; the implemented networks act on the available observations.

The plotted *episode return* is the undiscounted sum $\sum_{t=0}^{T-1} r_{t+1}$, whereas the learning targets use γ . Also, an episode is a trajectory; a gradient update is an optimization step. Neither should be called a training epoch in describing these experiments.

1.3 Why the Bellman operator is a contraction

Let $\mathcal{T}^*$ denote the right-hand side of Eq. (4) as an operator on bounded action-value functions.

Theorem 1.1 (Discounted Bellman contraction). *Assume bounded rewards, $\gamma < 1$, and a well-defined Bellman operator on the relevant complete space of bounded functions. Then*

$$\|\mathcal{T}^*Q_1 - \mathcal{T}^*Q_2\|_\infty \leq \gamma\|Q_1 - Q_2\|_\infty. \quad (5)$$

Thus $\mathcal{T}^*$ has a unique fixed point, and exact value iteration converges to it.

Proof. For each next state,

$$\left| \sup_a Q_1(s', a) - \sup_a Q_2(s', a) \right| \leq \sup_a |Q_1(s', a) - Q_2(s', a)|.$$

The reward terms cancel when subtracting the two Bellman updates. Taking the expectation, multiplying by γ , and then taking the supremum over (s, a) gives the inequality. Banach's fixed-point theorem gives the stated convergence. $\square$

This argument does not require the state space to be finite. The advantage of a small finite task is that its value function can be represented without approximation by a table. Exact representability does not mean that a table learned from finitely many samples is already exact. With neural networks, optimization, approximation error, and changing data distributions intervene between the ideal Bellman operator and the actual learning update. It is this distinction, rather than continuity alone, that limits the guarantees available for the robot agents [1].

2 Discrete environments

2.1 Shared tabular algorithms

Both discrete tasks maintain one value estimate per encoded state and action, initialized to zero. Exploration is ε -greedy: with probability ε , select uniformly from the actions; otherwise select an action maximizing the current estimate. The implementations use `argmax` for exploitation, so ties are resolved by the first maximizing action rather than randomized tie-breaking.

2.1.1 Q-learning: an off-policy target

For a transition $(s_t, a_t, r_{t+1}, s_{t+1})$, Q-learning updates

$$y_t^Q = r_{t+1} + \gamma b_t \max_{a'} Q_t(s_{t+1}, a'), \quad (6)$$

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)[y_t^Q - Q_t(s_t, a_t)]. \quad (7)$$

Only the visited entry changes. The bootstrap mask b_t is zero at a true terminal transition and one otherwise in the standard formulation. The treatment of time limits in the code is discussed separately below.

The maximum evaluates greedy continuation even when the next action actually executed is exploratory. This makes Q-learning *off-policy*: the policy being evaluated by the target differs from the behavior policy generating the trajectory.

Theorem 2.1 (Tabular Q-learning convergence [2]). *In a finite discounted MDP with bounded rewards, suppose every relevant nonterminal state–action pair is visited infinitely often and, for each such pair, the learning rates satisfy*

$$\sum_t \alpha_t(s, a) = \infty, \quad \sum_t \alpha_t(s, a)^2 < \infty. \quad (8)$$

Under the standard stochastic-approximation assumptions, tabular Q-learning converges almost surely to Q^ .*

The result permits a non-greedy behavior policy as long as it supplies sufficient coverage. It concerns the limiting value estimates and their greedy policy, not the return achieved while the behavior policy continues taking exploratory actions.

2.1.2 SARSA: learning from the behavior policy

SARSA first samples a_{t+1} from the behavior policy and uses

$$y_t^S = r_{t+1} + \gamma b_t Q_t(s_{t+1}, a_{t+1}), \quad (9)$$

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t(s_t, a_t)[y_t^S - Q_t(s_t, a_t)]. \quad (10)$$

In the standard SARSA trajectory, the sampled a_{t+1} is carried forward and executed on the next step. For a fixed policy, the conditional mean target contains $\sum_a \pi(a | s') Q(s', a)$, linking learning to that policy’s actual action probabilities. During control, both the estimates and the policy change.

Definition 2.1 (GLIE). A sequence of behavior policies is *greedy in the limit with infinite exploration* if relevant state–action pairs are visited infinitely often and the probability of selecting a non-greedy action tends to zero.

Theorem 2.2 (Tabular SARSA convergence [3]). *For a finite discounted MDP with bounded rewards, standard tabular SARSA converges almost surely to Q^* under GLIE exploration and the learning-rate conditions in Eq. (8), with the usual stochastic-approximation assumptions.*

Merely specifying $\varepsilon_t \rightarrow 0$ does not establish GLIE: exploration can disappear too quickly to ensure infinite visits. Conversely, maintaining a positive exploration floor violates the greedy-in-the-limit requirement. With persistent exploration, SARSA’s target continues to include exploratory behavior; this explains its different control objective without asserting unconditional convergence for a constant-step-size implementation.

2.1.3 What these guarantees mean for the project

Both training scripts use a constant learning rate $\alpha = 0.1$ and a finite budget of 5,000 episodes. Their exploration floors remain positive. In particular, a constant positive learning rate does not satisfy the square-summability condition over an infinite run. The algorithms’ theoretical guarantees therefore explain their design, but do not prove convergence or optimality of the reported finite experiments. Empirical improvement, a plateau, or success on a fixed map must be assessed on its own evidence.

2.1.4 Recorded discrete experiments

The four discrete training runs analyzed here completed on 13 September 2026, each with seed 0 and 5,000 episodes. They are the latest completed training runs for their environment and

Environment	Algorithm	Training transitions
Flag	Q-learning	338,791
Flag	SARSA	348,260
Cliff	Q-learning	357,234
Cliff	SARSA	333,769

Table 1: Actual budgets for the four 5,000-episode discrete runs. Each training transition produces one tabular update. Separate demonstrations and greedy evaluations are excluded.

Component	Implemented behavior
Observation	Agent coordinates and five binary values; 1 means the corresponding flag is still present.
Actions	0: stay; 1: up; 2: down; 3: left; 4: right.
Reward	+10 when entering a cell containing an uncollected flag; -0.5 otherwise. Collection does not also incur the step penalty.
End condition	All flags collected terminates immediately; otherwise the 100-step sampling cutoff truncates the rollout.
Encoding	Tuple $(r, c, f_1, \dots, f_5)$ indexing five Q-values.

Table 2: Flag environment, as configured by its training script.

algorithm, selected by timestamp rather than performance. Their saved settings match the configurations described below: learning rate 0.1, discounts 0.95 for flag collection and 0.99 for cliff walking, and external cutoffs of 100 and 200 steps respectively. Metadata identifies `complete-tabular-v2` state encoding and bootstrapping at cutoffs. The source directories are:

- Flag Q-learning: `artifacts/flag/qlearning/20260913T085943978379Z_seed0_85399cae/`.
- Flag SARSA: `artifacts/flag/sarsa/20260913T085951982917Z_seed0_d34cfb8b/`.
- Cliff Q-learning: `artifacts/cliff/qlearning/20260913T084122019817Z_seed0_8fb8d21b/`.
- Cliff SARSA: `artifacts/cliff/sarsa/20260913T084132538934Z_seed0_52f083d2/`.

The following figures are exact copies of the saved plots. Smoothing uses up to 100 preceding episodes; reward bands show rolling mean $\pm$ two rolling population standard deviations within one run, not confidence intervals across seeds. Such a band can extend beyond feasible episode returns; it is a dispersion summary, not a bound. Unless another window is stated, final training summaries use episodes 4,751–5,000, a 250-episode block, and therefore differ from plotted 100-episode endpoints. Training measurements include epsilon-greedy exploration and changing policies. Each final checkpoint was evaluated separately for one greedy episode with seed 1234 on its saved fixed task. Configuration records link each evaluation to its checkpoint; exact evaluation and figure paths are listed in `Report/figures/README.md`.

2.2 Flag collection environment

2.2.1 Environment specification

The agent starts at row–column coordinate $(5, 5)$ on a 10×10 grid. The five flags are placed at $(1, 2)$, $(3, 7)$, $(6, 1)$, $(7, 8)$, and $(9, 4)$, using zero-based coordinates. Flag positions are fixed; collecting one changes its status until the next reset. There are no internal obstacles in this environment. Attempted movements beyond the boundary are clamped to the nearest valid cell.

Algorithm	Mean return	Mean steps	Completion
Q-learning	37.63	29.74	250/250
SARSA	37.89	29.22	250/250

Table 3: Flag training outcomes over episodes 4,751–5,000. Both collect five flags in every episode, with no cutoffs in this block. These are training-policy results, not greedy evaluations.

The collection mask is essential: the best action at a location may change after a nearby flag has already been collected. Position alone would alias these situations. The encoding has at most $100 \cdot 2^5 = 3,200$ combinations and 16,000 state–action entries; this is a representational bound, not a claim that every combination is reachable. The implementation uses a dictionary of zero-initialized action-value arrays, allocated as states are accessed.

Collecting the final flag terminates on that same transition, with no additional action or step penalty. The rendering countdown from the earlier implementation has been removed from the transition logic. The 100-step limit is an external sampling cutoff rather than a task deadline; no remaining-time variable is required for that cutoff interpretation.

2.2.2 Training and implementation

The flag runner uses a shared tabular training loop. Q-learning and SARSA are each configured for 5,000 episodes with $\gamma = 0.95$ and $\alpha = 0.1$. For zero-based training episode e , exploration follows

$$\varepsilon_e = \max\left(0.05, 1 - \frac{0.95}{5000}e\right). \quad (11)$$

Decay occurs after each episode; the recorded epsilon is the value used during that episode. The default seed is 0, and evaluation selects greedy actions without changing the training exploration setting. Recording is optional: when enabled, a separate greedy demonstration is generated every 500 episodes. Run directories distinguish algorithms and seeds. A single run per algorithm still does not support a confidence interval across independently trained agents.

The SARSA loop selects the initial action before stepping. After a nonterminal transition it selects one next action, passes that action explicitly to `update_SARSA`, and carries it forward for execution. This now follows the standard SARSA trajectory; the former separate resampling inside the update has been removed. The cliff agent uses the same training logic.

Both updates mask the bootstrap only on `terminated`. A truncated transition retains its continuation value, while either flag stops the rollout. All three main environments use this same external-cutoff interpretation, as discussed in Section 4.

2.2.3 Training results and greedy evaluation

Figures 1 and 2 show a transition from almost uniformly cutoff-limited episodes to shorter, successful collection. During the first 250 episodes neither agent completes the task: both average 100 steps, with mean returns -33.49 for Q-learning and -33.96 for SARSA. They collect only 1.572 and 1.528 flags per episode on average. By episodes 2,251–2,500, Q-learning completes 228 of 250 episodes (91.2%), compared with 157 (62.8%) for SARSA. This establishes earlier reliable collection in the selected Q-learning run, rather than a universal difference in learning speed.

The outcome plots confirm that both agents achieve full collection in every final-block episode (Table 3). The final 100-episode plotted means are 37.79 return and 29.42 steps for Q-learning, and 38.005 return and 28.99 steps for SARSA. Their small final training-return difference does not establish an algorithm ranking from one seed.

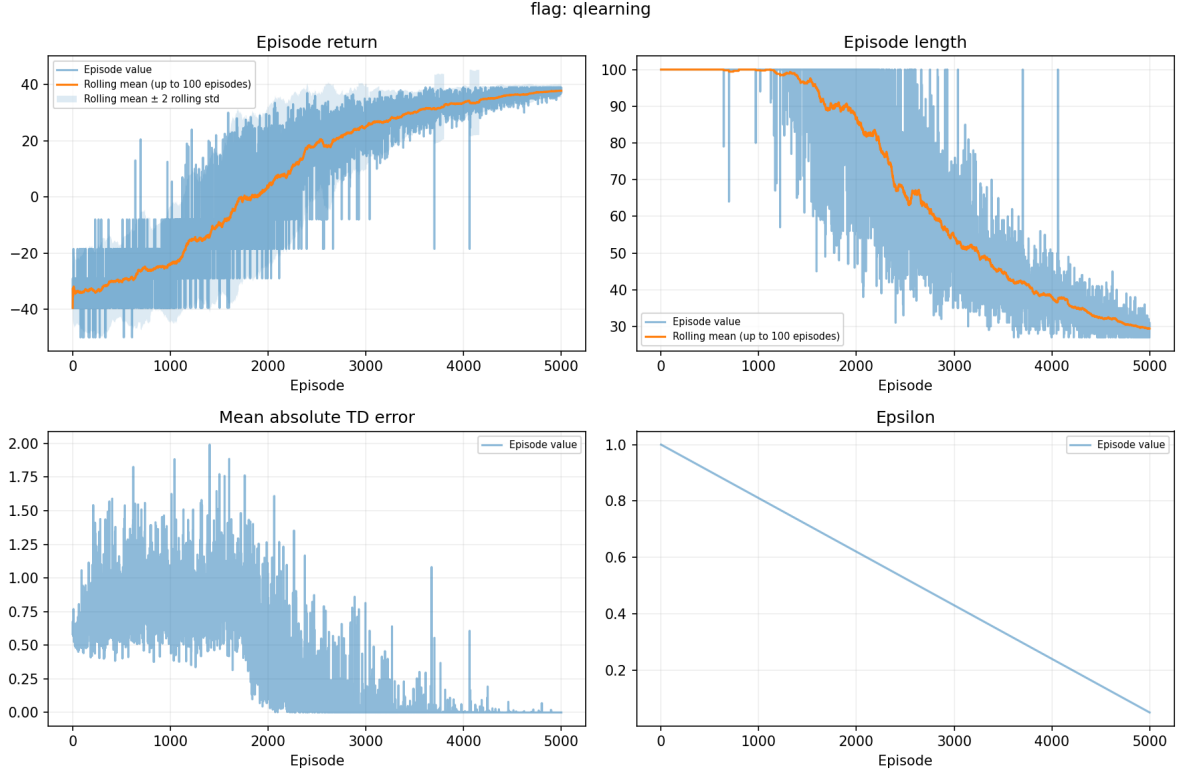


Figure 1: Flag Q-learning, 5,000 episodes with seed 0 and the corrected implementation. Return and length improve as training proceeds and exploration declines. The reward band is rolling mean $\pm$ two rolling standard deviations within this run.

The reward definition provides an exact interpretation rule. If an episode has T transitions and collects k flags, then its undiscounted return is

$$G^{\text{episode}} = 10k - 0.5(T - k) = 10.5k - 0.5T. \quad (12)$$

Here T ends immediately on final-flag collection or at the external cutoff. Early return improvements therefore reflect collecting more flags even while episodes still use the full cutoff. Once every episode collects all five, return becomes $52.5 - 0.5T$, so shorter episodes directly explain the final increase. This identity holds for every saved training row; the final means are consistent with the measured episode lengths.

The mean absolute TD-error panels answer a different question. Q-learning’s final-block mean is approximately 4.59×10^{-4} , whereas SARSA’s is 0.613. SARSA uses a sampled next action under continuing exploration, and its behavior-policy target also changes as epsilon decays. Different next actions can therefore leave variable sampled errors even when navigation succeeds. A small Q-learning TD error on visited transitions does not certify all table entries or prove an asymptotic convergence theorem.

In the saved greedy evaluations, both final checkpoints collect all five flags in 27 steps and obtain return 39.0, exactly $52.5 - 0.5 \cdot 27$. Replaying the saved tables without updates reproduces these records. Enumerating the $5! = 120$ flag orders and summing Manhattan distances from the fixed start gives a minimum of 27 steps; thus both evaluated routes attain the minimum transition count for this particular obstacle-free task. This finite geometric check does not establish optimal values at every encoded state, nor performance on different flag layouts.

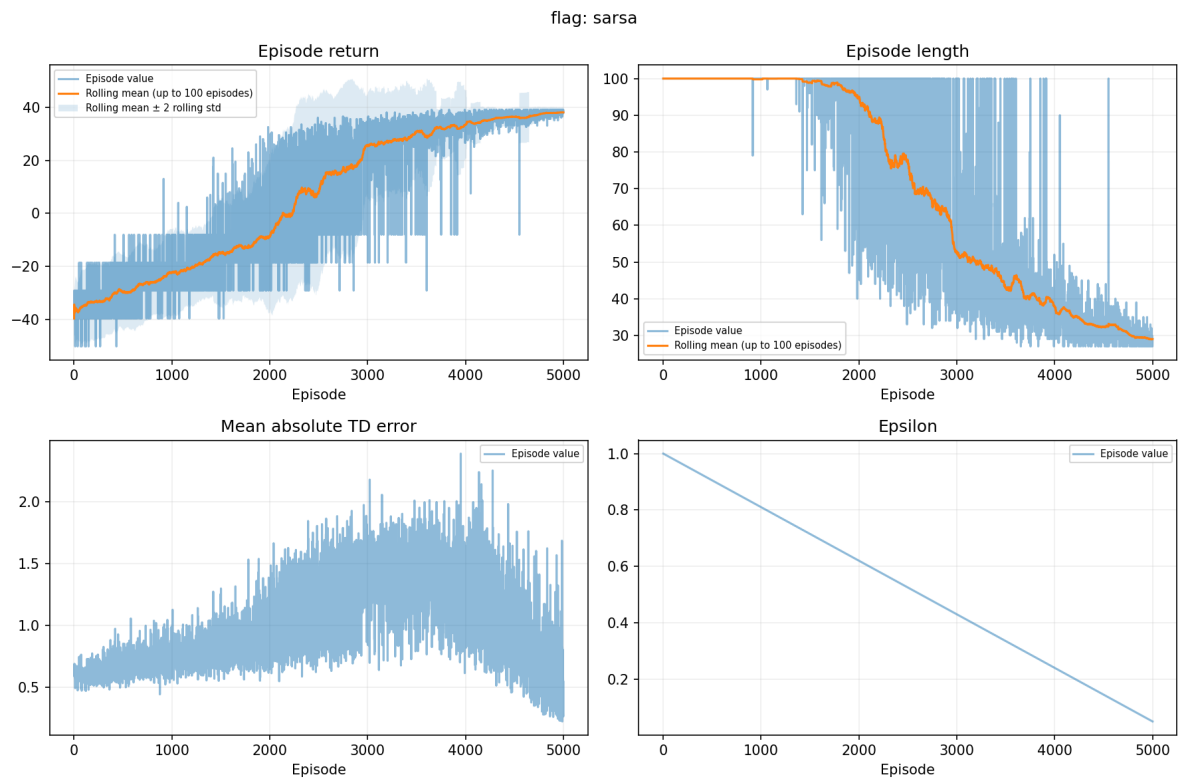


Figure 2: Flag SARSA under the same task, episode budget and exploration schedule. Successful collection becomes reliable later in this run than in the Q-learning run; sampled TD errors remain variable even after returns improve.

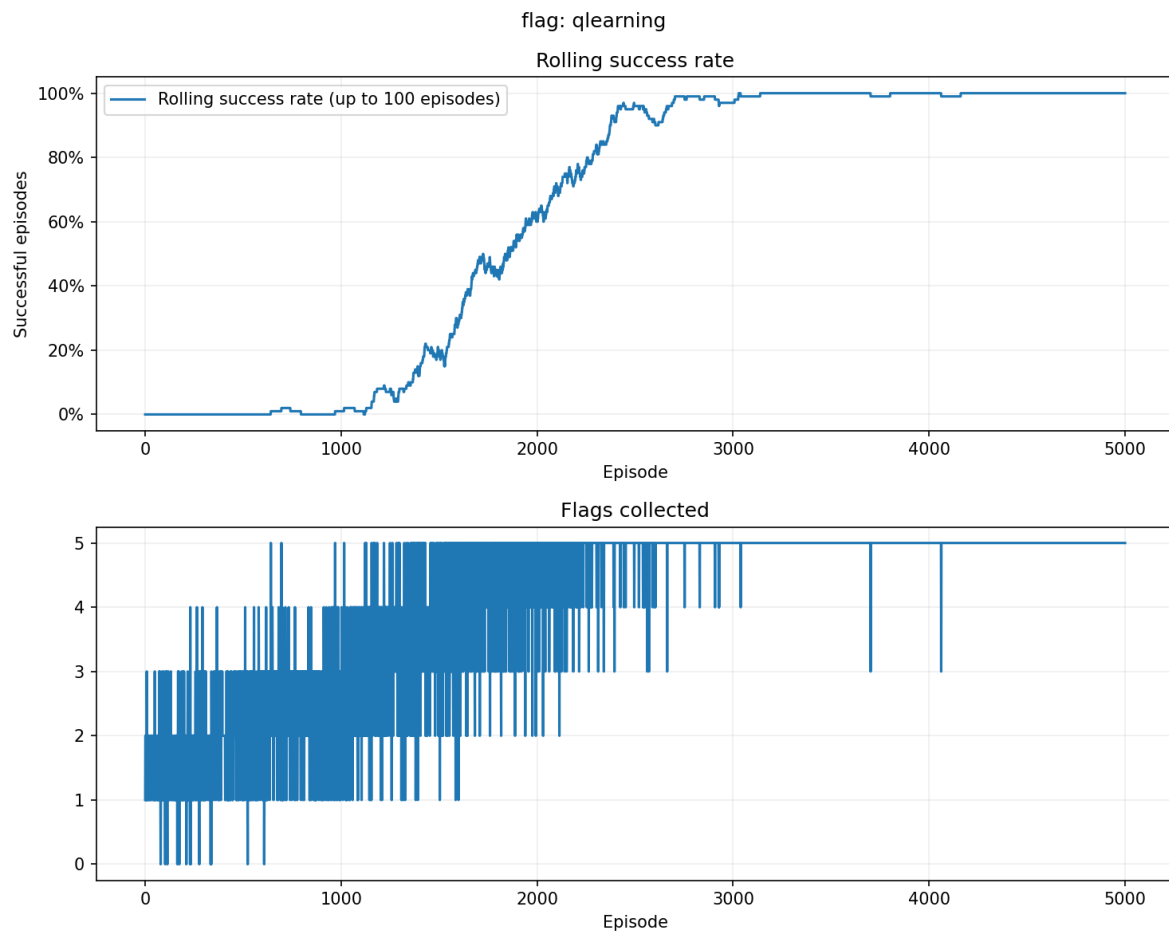


Figure 3: Flag Q-learning outcomes. The upper panel averages full-task completion over up to 100 episodes; the lower panel counts flags collected in each episode. Both distinguish partial collection from completing all five flags.

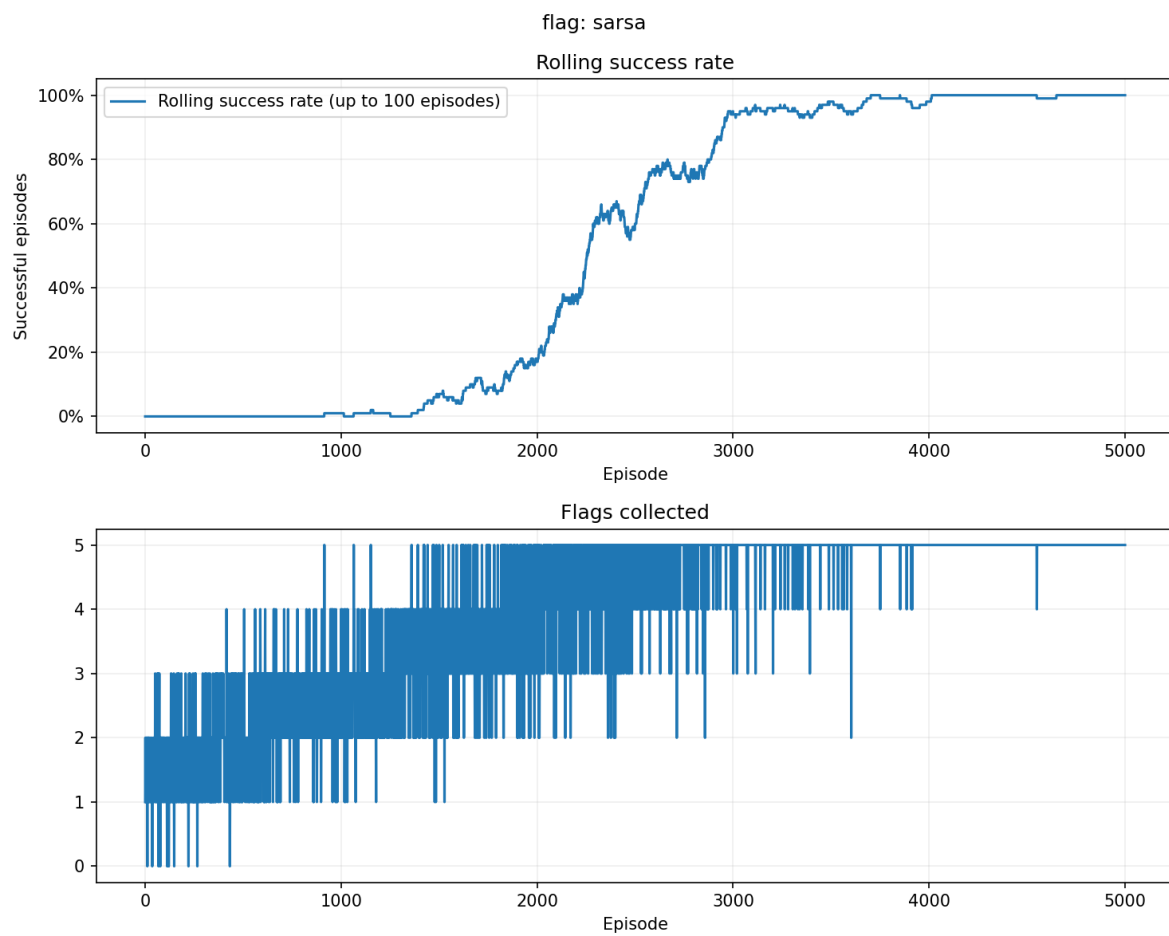


Figure 4: Flag SARSA outcomes. Occasional incomplete collections persist later in training, but all final 250 episodes collect every flag. Binary success indicators are not plotted.

2.3 Cliff walking environment

2.3.1 Environment specification

The active configuration is a canonical 5×16 cliff-walking grid. The start is $(4, 0)$ and the goal is $(4, 15)$. The fourteen cells between them on the bottom row form the cliff. Although the class and registration names mention random cliffs, the default layout used by the runner is **canonical**; the scattered-cliff generator is an optional alternative, not the active experiment.

The cutoff check is independent of the cliff-transition branch. A fall on step 200 returns the start observation and sets truncation, preserving the continuation bootstrap without allowing the rollout to exceed its limit. Goal arrival takes precedence over truncation.

2.3.2 Training configuration

Both algorithms use 5,000 episodes, $\alpha = 0.1$, and $\gamma = 0.99$. The training random generator is seeded with 0, and epsilon decays over the first 3,000 episodes:

$$\varepsilon_e = 1 - 0.9 \frac{\min(e, 3000)}{3000}. \quad (13)$$

The remaining episodes retain $\varepsilon = 0.1$. The shared training loop writes per-episode returns, lengths, success indicators, epsilon values and TD diagnostics to CSV, and saves its Q-table. The comparison routine smooths return, success and length over up to 100 episodes. Greedy evaluation runs one episode with seed 1234 per algorithm; optional recording captures that evaluation. Repeating the same deterministic fixed-map episode would not create independent evaluation scenarios.

The cliff SARSA loop chooses its next action before the update and executes that same action on the next step. Both algorithms suppress bootstrapping only for **terminated**, retaining a continuation value on truncation. In particular, a cliff fall must bootstrap from the start because the episode continues.

2.3.3 Why this task distinguishes SARSA from Q-learning

The shortest safe route in this grid moves up once, traverses fifteen columns immediately above the cliff, and moves down into the goal: 17 actions, with undiscounted return -17 in a perfect execution. This geometric reference is attained by the saved Q-learning checkpoint in greedy evaluation below. A route one row farther from the cliff can cost two additional actions but reduce exposure to an immediate cliff fall caused by an exploratory downward move.

Consider a state adjacent to the cliff and an epsilon-greedy policy whose greedy action is safe. With four actions and $\varepsilon = 0.1$, any particular exploratory action, including a downward move into the cliff where applicable, has probability $\varepsilon/4 = 0.025$. A fall costs -100 and returns the agent to the start, also losing progress toward the goal. Repeated falls can therefore produce

Component	Implemented behavior
Observation	A discrete position index $s = 16r + c$, with 80 declared indices. The returned position after a fall is the start.
Actions	0: up; 1: right; 2: down; 3: left. Boundary movements are clamped.
Ordinary transition	Reward -1 , including the transition into the goal.
Cliff transition	Reward -100 , immediate relocation to the start, and continuation of the same episode.
End condition	Goal arrival terminates; otherwise the configured 200-step cutoff truncates, including on a cliff fall.

Table 4: Active cliff-walking task. Falling is not a terminal event.

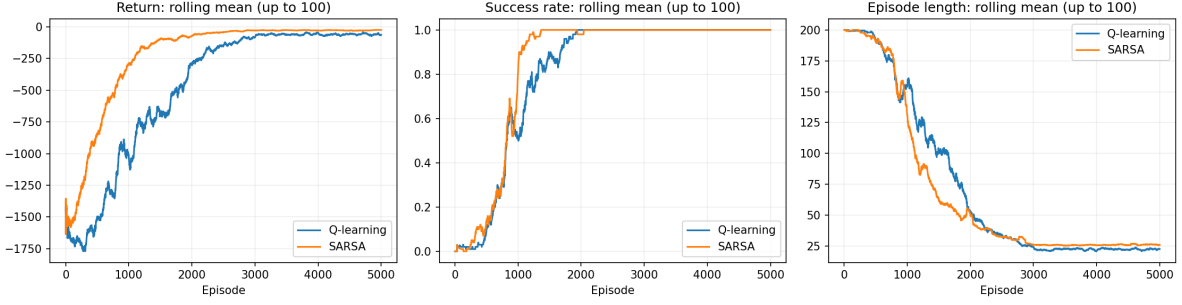


Figure 5: Cliff training comparison from the two completed 5,000-episode, seed-0 runs. SARSA achieves less negative training returns. Both eventually reach the goal reliably, while Q-learning has shorter late-training episodes but pays more cliff-fall penalties. The curves are rolling means, with no uncertainty bands across training seeds.

very negative returns within one episode.

The expected SARSA continuation target, conditional on a nonterminal next state s' , is

$$\mathbb{E}[y^S \mid s, a, r, s'] = r + \gamma \left[(1 - \varepsilon) \max_{a'} Q(s', a') + \frac{\varepsilon}{4} \sum_{a'} Q(s', a') \right]. \quad (14)$$

The low value of dangerous exploratory actions enters the average. By contrast, Q-learning uses only $r + \gamma \max_{a'} Q(s', a')$ for the same nonterminal transition. It learns about greedy continuation, under which a well-chosen action can follow the short path without deliberately entering the cliff. Actual falls are still learned from by Q-learning; the distinction is that the values of safe actions are backed up under greedy, rather than exploratory, future behavior.

The important difference is therefore a consequence of on-policy versus off-policy bootstrapping. SARSA can learn a longer route with better expected return while exploration remains active; Q-learning can learn the short cliff-adjacent greedy route yet obtain worse on-line training return because its behavior policy still explores. This is the mechanism illustrated by the classic cliff-walking example [1]. It is not a general guarantee that SARSA is intrinsically risk-averse or universally safer: it optimizes expected return under the action distribution represented in its target, without an explicit risk-sensitive objective.

Under greedy evaluation, exploratory falls disappear, so the comparison can favor the shorter learned route. If exploration eventually vanished with sufficient coverage and suitable learning rates, the standard tabular results would instead point toward the same optimal value function. The positive exploration floor in Eq. (13) deliberately keeps the training-policy distinction relevant.

2.3.4 Training results and the exploration tradeoff

Figure 5 provides empirical evidence for the predicted training-policy distinction. During the first 250 episodes, both agents reach the goal only four times (1.6%) and use almost the entire 200-step cutoff on average. Mean returns are -1697.10 for Q-learning and -1413.49 for SARSA. SARSA’s return improves sooner in the plotted run, while the success curves eventually approach one for both agents. Every episode in the final 2,000-episode exploration-floor phase completes successfully, yet their returns remain separated.

Success rate alone hides the cost of falling and recovering within a successful episode. With F falls during T transitions, the implemented reward gives the identity

$$G^{\text{episode}} = -T - 99F. \quad (15)$$

Algorithm	Mean return	Mean steps	Completion	Falls/episode
Q-learning	-62.88	22.48	250/250	0.408
SARSA	-26.91	26.12	250/250	0.008

Table 5: Cliff training outcomes over episodes 4,751–5,000, with $\varepsilon = 0.1$ for both agents. Fall counts are derived from Eq. (15); all episodes end at the goal.

Algorithm	Return	Steps	Greedy route
Q-learning	-17	17	Across row 3, immediately above the cliff.
SARSA	-23	23	Up to row 0, across the top, then down to the goal.

Table 6: Saved greedy evaluations of the final cliff checkpoints, one episode each with seed 1234. Both reach the goal without falling. Route descriptions were verified by replaying the saved Q-tables without learning updates. Coordinates are zero-based.

Thus $F = (-G^{\text{episode}} - T)/99$ recovers an exact nonnegative integer fall count from each saved return/length pair. These counts concern nonterminal cliff falls; they are not robot collision indicators.

In this final block Q-learning falls 102 times across 71 episodes, whereas SARSA falls twice across two episodes. SARSA uses 3.636 more steps per episode on average, but the reduction in penalties raises its mean return by 35.964. This is the observed training-return advantage of accounting for future exploratory actions. It also persists over the full final 2,000 episodes: mean returns are -61.01 and -28.86, with 781 and 59 falls for Q-learning and SARSA respectively. The final 100-episode endpoints in the figure are -65.22 and -25.96; they use a shorter window than Table 5.

Greedy evaluation reverses the return ordering (Table 6): Q-learning takes the shortest safe route, while SARSA retains a six-step detour farther from the cliff. With exploration disabled, the short route suffers no accidental downward moves and earns the higher return. The training fall counts and verified routes therefore support the on-policy/off-policy explanation in this particular experiment. One trained seed and one fixed map do not establish a universal safety or performance ranking.

3 Continuous differential-drive navigation

3.1 Environment, dynamics, and observations

The shared `DiffDriveEnv` simulates a circular robot in a 10×10 metre room containing axis-aligned rectangular obstacles. In the training notebooks the robot starts at (1,1), initially facing angle zero, and aims for (8.5, 8.5). Obstacles remain static within an episode. The robot has radius 0.3 m and a simulation step of $\Delta t = 0.1$ s. The notebook settings override some constructor defaults, notably the goal position and episode horizon.

The control action is $a_t = (v_t, \omega_t)$, with

$$v_t \in [-0.5, 1.0] \text{ m/s}, \quad \omega_t \in [-\pi, \pi] \text{ rad/s}. \quad (16)$$

These are linear and angular velocities, not separate wheel-speed commands. The discrete

kinematic update first changes the heading and then advances along the new heading:

$$\theta_{t+1} = \text{wrap}_{[-\pi, \pi)}(\theta_t + \omega_t \Delta t), \quad (17)$$

$$\tilde{x}_{t+1} = x_t + v_t \cos(\theta_{t+1}) \Delta t, \quad (18)$$

$$\tilde{y}_{t+1} = y_t + v_t \sin(\theta_{t+1}) \Delta t. \quad (19)$$

The candidate position is checked against room walls and obstacles using the robot radius. On collision the translation is rejected and the episode terminates. Otherwise the candidate position is accepted. The goal is reached when the distance to it is strictly less than $0.3 + 0.2 = 0.5$ m.

Sixteen evenly spaced LiDAR rays rotate with the robot heading and report distances to walls or obstacles, capped at 5 m. The observation is

$$o_t = (\ell_{t,1}, \dots, \ell_{t,16}, d_t, \phi_t) \in \mathbb{R}^{18}, \quad (20)$$

where d_t is goal distance and $\phi_t \in [-\pi, \pi)$ is the relative goal bearing. Absolute (x, y) coordinates are not included. The two-dimensional action space is continuous; it should not be described as high-dimensional merely because it has infinitely many possible values.

The full simulator state includes the robot pose and obstacle layout. Sparse finite-range sensing can produce similar observations in different global configurations. The agents use memoryless policies on o_t , without a map or recurrent state; describing Eq. (20) as an exactly observed MDP state would therefore be too strong. The observation omits time because the rollout length is an external sampling cutoff; sparse sensing still makes the full simulator state only partially observed.

3.1.1 Obstacle generation

With `random_obst=True`, a new obstacle configuration is sampled on reset. The mode named `curriculum` selects between three families with fixed proposal probabilities: perturbed detour layouts (0.4), walls with a gap (0.4), and random rectangular blocks (0.2). Candidate layouts are checked for clearance around the start and goal and for a feasible path on a discretized collision-aware grid. Rejection and fallback logic can affect the distribution of accepted layouts.

This is a mixture of navigation scenarios, not a schedule of increasing difficulty. The fixed evaluation map is also the base geometry of the perturbed-detour generator. Success on that map is therefore evidence of a useful navigation behavior within the design family, rather than a strong test of generalization to an unrelated obstacle distribution.

3.1.2 Active reward function

For a nonterminal transition, let d_{t+1} and ϕ_{t+1} be the new distance and relative bearing. Define c_{t+1} as the minimum LiDAR reading, f_{t+1} as the minimum reading in the forward $\pm 45^\circ$ sector, and g_{t+1} as the reading on the ray closest to the goal direction. The normalized danger terms are

$$D_{t+1} = \max\left(0, \frac{0.5 - c_{t+1}}{0.5}\right), \quad D_{t+1}^f = \max\left(0, \frac{1.0 - f_{t+1}}{1.0}\right). \quad (21)$$

These use ray distances from the robot centre, rather than exact clearance from the robot's boundary. The progress weight is

$$w_{t+1} = \begin{cases} 1, & g_{t+1} < 0.8 \text{ m}, \\ 10, & \text{otherwise.} \end{cases} \quad (22)$$

The active dense reward is

$$r_{t+1}^{\text{dense}} = w_{t+1}(d_t - d_{t+1}) - 0.5D_{t+1}^2 - 0.5(D_{t+1}^f)^2 + 0.3 \left(1 - \frac{|\phi_{t+1}|}{\pi} \right) - 0.02|\omega_t| - 0.5. \quad (23)$$

This rewards progress and orientation toward the goal, penalizes proximity to obstacles and unnecessary turning, and charges a time cost. Lowering the progress weight when the goal direction is obstructed reduces the immediate cost of detouring, although heading guidance still favors the goal direction.

The final reward and episode flags follow the priority order

$$r_{t+1} = \begin{cases} -100, & \text{collision (termination),} \\ +100, & \text{goal reached without collision (termination),} \\ r_{t+1}^{\text{dense}}, & \text{otherwise.} \end{cases} \quad (24)$$

Terminal rewards replace dense shaping on that step. Reaching the sampling limit without a terminal event produces truncation and the ordinary dense reward: there is no additional timeout penalty. Collision or goal termination takes precedence. The danger-reduction term remains inactive because its coefficient is zero. The selected new training runs record this reward as **navigation-cutoff-v2**; the archived historical benchmark belongs to an earlier execution setup.

Reward design affects which behavior is optimal. The state-dependent progress weight and the additional penalties are not established here as policy-invariant potential-based shaping. Therefore the objective being learned is the complete shaped reward, rather than an independently proven shortest-path or minimum-collision objective.

3.2 Shared deep-learning implementation

All three agents are model-free, off-policy actor-critic methods implemented in PyTorch. An actor selects actions and one or more critics estimate action values. Replay buffers retain raw observations, actions, rewards, next observations, and a terminal mask; uniformly sampled minibatches reuse transitions collected by earlier policies. DDPG and TD3 use the NumPy replay buffer, while SAC uses the tensor-based replay buffer. SAC’s storage is on the CPU, and sampled batches are moved to the configured device.

Before network evaluation, the current implementation normalizes LiDAR readings by 5, distance by the room diagonal $\sqrt{10^2 + 10^2}$, and relative bearing by π . Thus most components lie in $[0, 1]$, and bearing lies in $[-1, 1]$. The same transformation is applied when choosing actions and when learning from replay. It is a fixed physical scaling, not a fitted running normalization.

The deterministic actor used by DDPG and TD3 has two fully connected ReLU hidden layers followed by a two-dimensional tanh output. If $u \in [-1, 1]^2$ is that output, physical actions are obtained componentwise as

$$a = \frac{a_{\max} - a_{\min}}{2} \odot u + \frac{a_{\max} + a_{\min}}{2}. \quad (25)$$

DDPG’s single critic first processes the observation through a hidden layer, concatenates the action, and applies another hidden layer before the scalar output. TD3 and SAC instead use two independent critics, each taking the concatenated observation and action through two ReLU hidden layers. The network widths are reported in Table 7.

Adam optimizers update network parameters. The current constructors set the actor and critic gradient-norm bound to 1.0. Target networks start as copies of the online networks and are

updated by

$$\bar{\psi} \leftarrow \tau\psi + (1 - \tau)\bar{\psi}, \quad (26)$$

with the analogous operation for target actors where present. A small τ slows the target’s response to online updates. Replay, target networks, scaling, and clipping are practical stabilization mechanisms; none proves global convergence.

Training begins with uniformly sampled random actions. Learning starts after the warmup threshold and once a minibatch is available. Replay stores only **terminated** in its bootstrap mask, while either termination or truncation stops a rollout. Observations are normalized as tensors on the learning device. Action selection disables gradient tracking; critic parameters are frozen during actor optimization while gradients still propagate through critic inputs to the actor. Per-episode loss summaries replace unbounded per-update histories.

3.3 DDPG: deterministic actor and a single critic

DDPG combines a deterministic policy μ_θ with an action-value critic Q_ψ [5]. The deterministic policy gradient theorem [4] supplies its motivation. For a differentiable policy and sufficiently regular dynamics and value functions, the exact policy gradient can be written

$$\nabla_\theta J(\mu_\theta) = \int \rho^{\mu_\theta}(s) \nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s, a) \big|_{a=\mu_\theta(s)} ds, \quad (27)$$

where ρ^{μ_θ} is the unnormalized discounted state-visitation measure for the start distribution. This identity uses the true action-value function and the appropriate occupancy measure. Replacing them with a learned critic and replay samples is a practical approximation, not the same exact theorem.

For a replay transition (z, a, r, z', m) , with $m = 1$ for a true terminal event, the implementation uses

$$y^{\text{DDPG}} = r + \gamma(1 - m)Q_{\bar{\psi}}(z', \mu_{\bar{\theta}}(z')), \quad (28)$$

$$L_Q(\psi) = \mathbb{E}_{\mathcal{D}} [(Q_\psi(z, a) - y^{\text{DDPG}})^2], \quad (29)$$

$$L_\mu(\theta) = -\mathbb{E}_{z \sim \mathcal{D}} [Q_\psi(z, \mu_\theta(z))]. \quad (30)$$

The target is held fixed during the critic update. The actor is optimized through the critic’s action gradient, and both target networks are softly updated after each learning update.

After warmup, DDPG adds temporally correlated Ornstein–Uhlenbeck (OU) exploration noise to physical actions. The configured noise standard-deviation parameter is 0.2, with each noise component clipped to $[-0.5, 0.5]$; the resulting action is then clipped to the environment bounds. The OU state is reset each episode. Evaluation disables this noise.

A single critic can overestimate actions that the actor then exploits. Approximation error, bootstrapping, and off-policy data form the combination commonly called the *deadly triad* [1]. An attractive predicted action can be poor in the environment, and changing actor and critic parameters can destabilize each other. DDPG with these nonlinear networks has no general convergence guarantee; even convergence to a local optimum of the true return is not assured. Loss curves must therefore be interpreted alongside navigation outcomes.

3.4 TD3: reducing actor–critic estimation error

TD3 retains deterministic actions and replay learning but modifies DDPG in three ways [6]. First, it uses two critics and takes the smaller target estimate. Second, it delays actor and target-network updates relative to critic updates. Third, it smooths the target policy with clipped Gaussian noise.

For a sampled transition, let

$$\epsilon \sim \mathcal{N}(0, \sigma_{\text{target}}^2 I), \quad \tilde{\epsilon} = \text{clip}(\epsilon, -c, c), \quad (31)$$

$$\tilde{a}' = \text{clip}(\mu_{\tilde{\theta}}(z') + \tilde{\epsilon}, a_{\min}, a_{\max}), \quad (32)$$

$$y^{\text{TD3}} = r + \gamma(1 - m) \min_{j=1,2} Q_{\tilde{\psi}_j}(z', \tilde{a}'). \quad (33)$$

The two critics minimize the sum of their squared errors to this target. On each delayed actor update, the actor minimizes

$$L_{\mu}(\theta) = -\mathbb{E}_{z \sim \mathcal{D}}[Q_{\psi_1}(z, \mu_{\theta}(z))]. \quad (34)$$

The actor uses the first critic, rather than the minimum of both. Both target critics and the target actor are softly updated only on these delayed updates.

The implementation sets $\sigma_{\text{target}} = 0.2$, $c = 0.5$, and an actor-update delay of two critic updates. Target smoothing acts inside the Bellman target; it is distinct from behavior-policy exploration. This project’s TD3 agent uses OU exploration with the same 0.2 scale and 0.5 clipping bound as DDPG, while target smoothing uses Gaussian noise. The training notebook requests two critic updates per environment step after warmup, giving one actor/target update per step under that delay.

The minimum target reduces optimistic errors, delay gives the critics additional updates before the actor changes, and smoothing discourages exploitation of narrow erroneous value peaks. These mechanisms can improve stability, but the minimum can also introduce underestimation and the critics can have correlated errors. TD3 does not remove approximation error or provide a general proof of convergence to an optimal navigation policy.

3.5 SAC: stochastic control and entropy regularization

SAC augments return with policy entropy [7, 8]. To distinguish its temperature from the tabular learning rate, denote the temperature by $\eta > 0$ (called `alpha` in the code):

$$J_{\eta}(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_{t+1} + \eta \mathcal{H}(\pi(\cdot | s_t))) \right]. \quad (35)$$

Entropy encourages a distribution of actions rather than immediate commitment to a single action. In continuous spaces this is differential entropy with respect to the chosen action coordinates; it need not be positive.

3.5.1 Soft policy iteration and its limits

For fixed η and policy π , define

$$V_{\eta}^{\pi}(s) = \mathbb{E}_{a \sim \pi} [Q_{\eta}^{\pi}(s, a) - \eta \log \pi(a | s)], \quad (36)$$

$$(\mathcal{T}_{\eta}^{\pi} Q)(s, a) = \mathbb{E} [r + \gamma \mathbb{E}_{a' \sim \pi} [Q(s', a') - \eta \log \pi(a' | s')]]. \quad (37)$$

For a fixed policy with finite entropy terms, differences of two such updates cancel the entropy contribution. The evaluation operator is then a γ -contraction. In the finite-action, unrestricted policy case, exact soft improvement gives

$$\pi_{\text{new}}(a | s) = \frac{\exp(Q_{\eta}^{\pi_{\text{old}}}(s, a)/\eta)}{\sum_b \exp(Q_{\eta}^{\pi_{\text{old}}}(s, b)/\eta)}. \quad (38)$$

Theorem 3.1 (Exact soft policy iteration, finite tabular case). *With bounded rewards, finite state and action sets, $\gamma < 1$, and fixed positive temperature, exact soft policy evaluation and unrestricted soft policy improvement produce nondecreasing soft values converging to the entropy-regularized optimum [7].*

This statement concerns the entropy-regularized objective, not expected reward alone. The implemented continuous SAC algorithm uses parametric policies, approximate minibatch optimization, and an adapted temperature; it does not perform the exact operations in the theorem. The theorem therefore does not become an approximate or asymptotic convergence guarantee for the neural implementation.

3.5.2 Implemented actor, critics, and temperature

The Gaussian actor predicts a mean and log standard deviation from two ReLU hidden layers. Log standard deviations are clipped to $[-20, 2]$. A reparameterized Gaussian sample is passed through tanh and rescaled to the physical action bounds. Its log density includes the tanh and action-scaling Jacobian corrections. Deterministic evaluation applies the same transformation to the Gaussian mean; this is not generally the expectation of the transformed random action.

SAC has two online critics and two target critics, with no target actor. Its update equations are

$$a' \sim \pi_\theta(\cdot \mid z'), \quad (39)$$

$$y^{\text{SAC}} = r + \gamma(1 - m) \left[\min_j Q_{\bar{\psi}_j}(z', a') - \eta \log \pi_\theta(a' \mid z') \right], \quad (40)$$

$$L_Q = \mathbb{E}_{\mathcal{D}} \left[\sum_{j=1}^2 (Q_{\psi_j}(z, a) - y^{\text{SAC}})^2 \right], \quad (41)$$

$$L_\pi = \mathbb{E}_{z \sim \mathcal{D}, a \sim \pi_\theta} [\eta \log \pi_\theta(a \mid z) - \min_j Q_{\psi_j}(z, a)]. \quad (42)$$

Gradients for the policy flow through reparameterized actions. Automatic entropy tuning starts from $\eta = 0.2$, uses target entropy $\mathcal{H}_{\text{target}} = -2$ for two action dimensions, and learns $\beta = \log \eta$ with

$$L_\beta = -\mathbb{E} [\beta \text{ stopgrad}(\log \pi_\theta(a \mid z) + \mathcal{H}_{\text{target}})]. \quad (43)$$

This is the log-temperature loss used in the code. The target critics are softly updated after each learning update. A stochastic policy and twin critics provide mechanisms for exploration and reduced overestimation, but they do not establish that SAC will outperform the other agents on this project. The temperature also need not decrease monotonically during learning.

3.6 Training configuration and comparison protocol

Table 7 records the settings saved with the three completed runs analyzed below. The active training notebooks expose environment and agent parameters directly, so DDPG, TD3 and SAC can be configured independently. Each run’s `config.json` is an automatic snapshot of the effective settings, not a shared input configuration that overrides the notebook.

The selected artifacts are the latest completed training run for each algorithm in the supplied project, chosen by run timestamp rather than by return. All three were created on 10 September 2026. Their directories beneath `artifacts/continuous/` are:

- DDPG: `ddpg/20260910T154140101411Z_seed0_3d2d5e39/`.
- TD3: `td3/20260910T205530372576Z_seed0_2e48e740/`.
- SAC: `sac/20260910T181357962640Z_seed0_88a0e82c/`.

Each directory contains a completed configuration record, a full sequential episode CSV and its plots. Metadata identifies `physical-observation-v1` preprocessing, `navigation-cutoff-v2` reward, bootstrapping at external cutoffs, and no source checkpoint: these are fresh runs, not

Parameter	DDPG	TD3	SAC
Completed training episodes	5,000	5,000	3,000
Maximum steps per episode	1,000	1,000	1,200
Actor learning rate	10^{-4}	10^{-4}	$3 \cdot 10^{-4}$
Critic learning rate	10^{-3}	10^{-3}	$3 \cdot 10^{-4}$
Discount γ	0.995	0.99	0.99
Target update τ	0.001	0.005	0.005
Hidden width	256	256	64
Replay capacity	300,000	300,000	100,000
Minibatch size	256	256	256
Random-action warmup steps	5,000	5,000	5,000
Critic updates per environment step after warmup	1	2	2
Actor update interval (critic updates)	1	2	1
Actor/critic gradient-norm bound	1.0	1.0	1.0
CSV logging interval (episodes)	1	1	1
Notebook summary interval (episodes)	500	500	500
Demonstration interval (episodes)	500	500	500

Table 7: Settings verified in the selected completed runs. All three use seed 0, CPU computation, physical observation normalization, and the corrected reward without an extra timeout penalty.

Algorithm	Environment steps	Critic updates	Actor updates
DDPG	1,230,920	1,225,921	1,225,921
TD3	2,532,894	5,055,790	2,527,895
SAC	673,498	1,336,998	1,336,998

Table 8: Actual cumulative training budgets from the final CSV row of each selected run. A TD3 or SAC critic update optimizes both critics; these are optimizer-update counts, not individual-network counts. Demonstration rollouts are excluded from the learning-step budget.

warm starts. The additional earlier DDPG run uses the same seed and is not counted as an independent replication. The figure inventory records source paths and SHA-256 hashes for the copied images and numerical inputs.

SAC uses automatic temperature tuning with learning rate $3 \cdot 10^{-4}$ and initial temperature 0.2. Training runs only when the notebook’s explicit `RUN_TRAINING` flag is enabled; skipping that cell does not load an old model. All three selected runs have now completed, including SAC with normalized inputs.

Checkpoints identify architecture, physical input scaling, action bounds, reward version and source configuration. Evaluation and warm-start loading are explicit: a warm start keeps network weights but resets replay, optimizers, exploration, temperature and counters, rather than claiming exact resumption. Missing or incompatible checkpoints are reported as unavailable for comparisons.

Equal episode counts do not imply equal sample or optimization budgets. TD3 collects substantially more transitions than DDPG because its episodes are longer on average; SAC has a shorter episode budget but a larger cutoff and a smaller network. Discounts, target update speeds, replay capacities, behavior policies and update frequencies also differ. Table 8 therefore describes the resources consumed by these individual experiments, not a controlled test of algorithm efficiency.

The optional controlled-comparison workflow uses training seeds 0–4, a common budget of 1.5 million environment steps per agent, and a 1,000-step sampling cutoff. It stores the same 51

evaluation scenarios for all agents: the historical fixed map and 50 mixture-generated layouts with seeds 10000–10049. These additional layouts come from the same proposal families, not a claimed out-of-distribution test. Agent-specific widths and optimizer settings remain explicit, and actual critic and actor update counts are recorded. Aggregation reports variation across training seeds separately from within-run variability. This full controlled comparison has not been run for the report.

Optional videos are separate greedy demonstrations, reduced to at most 320 pixels wide and capped at 300 frames. Evaluation continues to termination or truncation after recording stops. Names such as `episode_0.mp4` and `episode_500.mp4` use zero-based training indices, and a demonstration follows the corresponding training episode’s updates. Thus video 0 follows the first episode; CSV episode numbers and plot axes count completed episodes from 1. Notebook summaries retain each block of 500 completed episodes above the updating total progress bar.

3.7 Training results and interpretation

The nine figures in this subsection are copied without alteration from the three completed corrected-code runs identified in Section 3.6. Their measurements are training outcomes under exploration: DDPG and TD3 use OU noise after warmup, whereas SAC samples its stochastic policy. They are not greedy evaluation curves or repeated measurements of only the final checkpoint. Numerical claims below are recomputed from the corresponding episode CSVs.

The figures use rolling windows of up to 100 episodes. For completed-episode count e , let $n_e = \min(100, e)$. The reward mean and population standard deviation are

$$\bar{G}_e = \frac{1}{n_e} \sum_{i=e-n_e+1}^e G_i, \quad s_e = \sqrt{\frac{1}{n_e} \sum_{i=e-n_e+1}^e (G_i - \bar{G}_e)^2}. \quad (44)$$

All three reward plots shade $\bar{G}_e \pm 2s_e$. This is within-run variability, not a confidence interval across independent training seeds; the band may extend beyond the observed return range. Success curves average the binary goal-completion flags over the same windows. Individual success and collision indicators are not drawn. Losses are episode averages over the updates performed during that episode, with missing values when no update occurred.

The separate square distance plots show the distance at the end of every episode, including both successes and failures. Their 0–10 m vertical range is a display choice, not a bound on the room diagonal. Triangles at the top mark episodes whose final distance exceeds 10 m; the rolling mean and reported statistics use the original, unclipped values. Unless explicitly described as a plotted rolling value, the numerical summaries in the text and Table 9 use the preceding 250 episodes. This distinction matters when comparing a quoted number with the endpoint of a plotted 100-episode curve.

3.7.1 DDPG: strong improvement with temporary regressions

DDPG improves substantially after a difficult initial phase. Mean return over episodes 1–250 is -104.65 , with 24.8% success and mean final distance 5.02 m. Over the 250 episodes ending at episode 1,000, return is 21.81 and success is 65.2%; at episode 2,000, they reach 103.32 and 85.6%. The final-distance curve and the declining mean episode length support the interpretation that the agent increasingly reaches the goal in shorter trajectories.

The improvement is not monotonic. The window ending at episode 2,750 has mean return 133.54 and success 90.4%; the following 250-episode window falls to 88.17 and 78.4%, with mean final distance rising from 0.90 to 1.47 m. This is a regression around episode 3,000 in the selected run, rather than the different decline described by the old pre-fix figures. By the window ending at 3,750, return recovers to 140.56 with 92.8% success, followed by further variability.

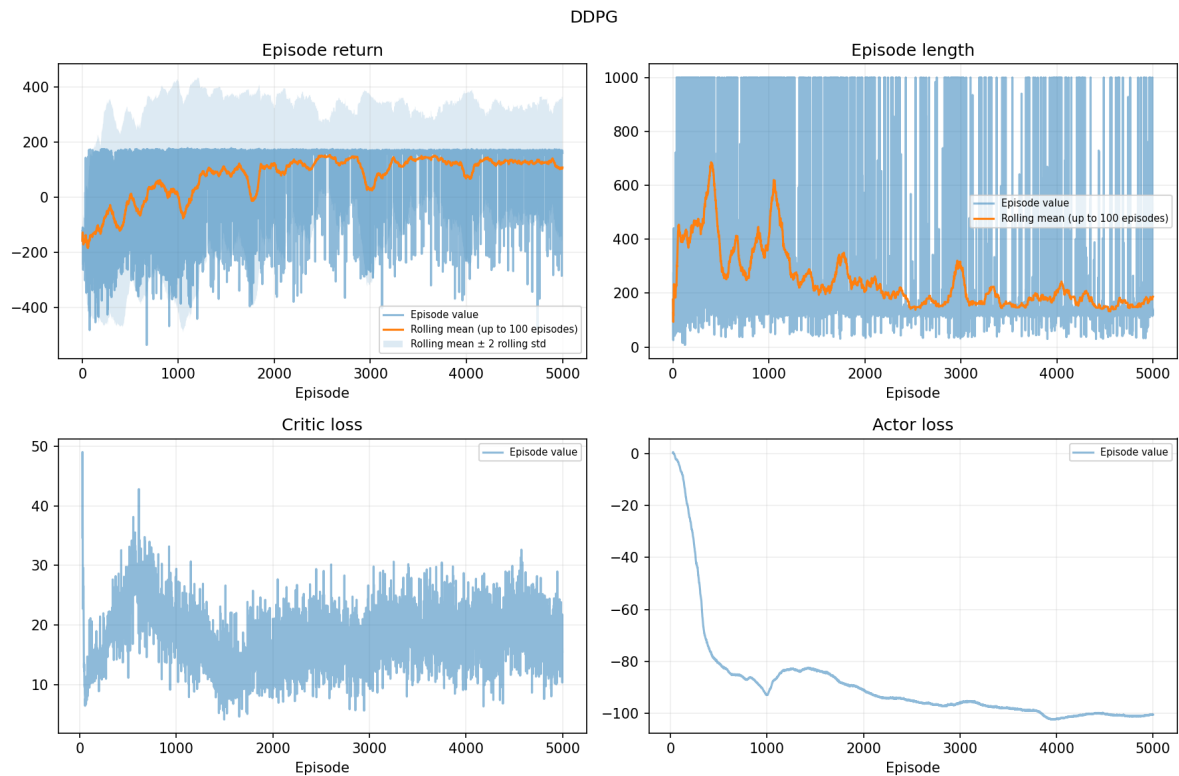


Figure 6: Selected DDPG run over 5,000 completed episodes. Return and length show individual episodes and rolling means over up to 100 episodes. The reward band is rolling mean $\pm$ two rolling standard deviations within this run. Actor and critic losses are episode-averaged optimization diagnostics.

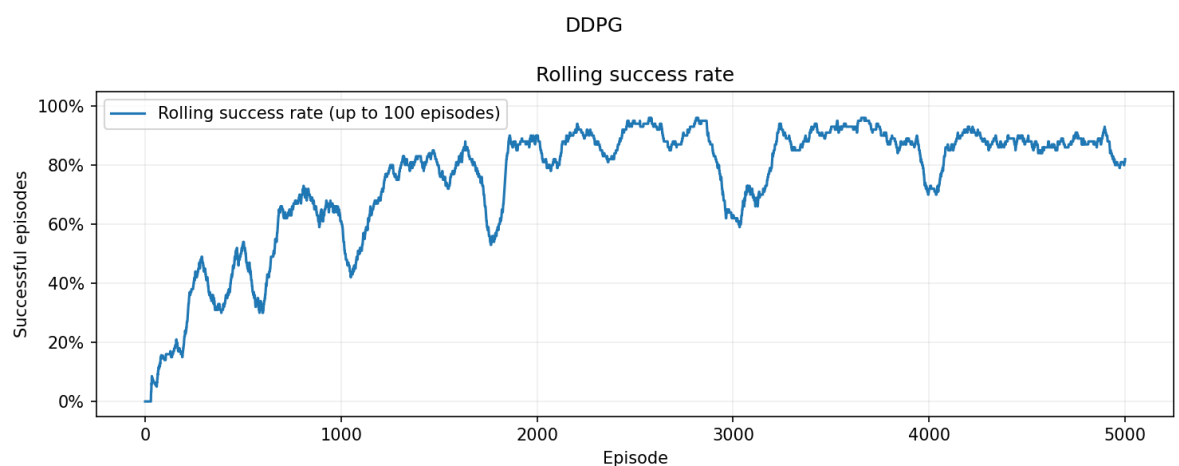


Figure 7: DDPG success rate over up to 100 recent training episodes. Success is determined by goal completion, independently of the sign of the shaped return. The final plotted value is 82%, whereas the final 250-episode success fraction is 85.2%.

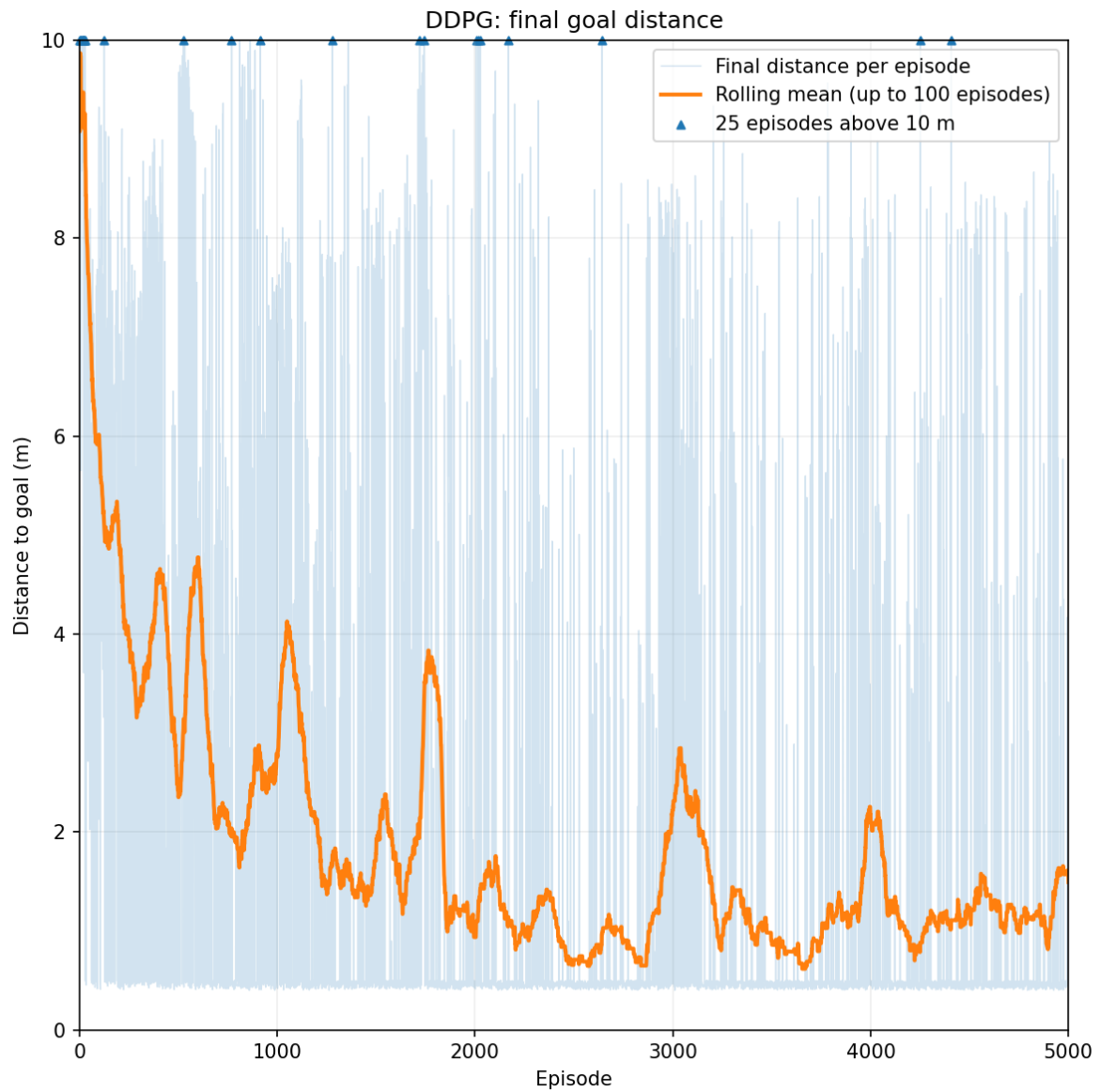


Figure 8: DDPG final goal distance in metres for all training episodes, with a rolling mean over up to 100 episodes. Twenty-five episodes exceed the displayed 10 m limit and are marked at the upper edge. The mean is computed from unclipped distances.

In the final 250 episodes, mean return is 118.49, mean length is 169.04 steps, and mean final distance is 1.29 m. There are 213 successes, 28 collisions and nine cutoffs, giving an 85.2% training success rate. The shorter 100-episode endpoint is 82%, so the last part of the run does not establish a stable success probability above 90%. The remaining failures and occasional long episodes are visible even when the rolling return is positive.

The actor loss approaches roughly -100 while the critic loss remains nonzero and variable. A more negative actor loss reflects the critic’s predicted values for the chosen actions; it is not itself a measured navigation score. Exploration, sampled layouts and changing networks all vary during training, so these figures do not isolate a cause for the temporary regressions.

3.7.2 TD3: *limited improvement and persistent cutoffs*

The new TD3 experiment does not reproduce the strong late-training performance of the historical run. Mean return improves from -79.63 in episodes 1–250 to -4.69 in the window ending at episode 1,000, with success increasing from 32.0% to 56.4%. Subsequent returns and success rates fluctuate without a sustained move to high reliability. For example, the window ending at episode 2,000 has mean return -7.05 and success 54.4%.

The episode-averaged critic loss reaches 3,674.03 at episode 2,530, and the actor loss reaches 108.84 at episode 2,553. The window ending at 2,750 has mean return -53.00 , success 41.6%, and mean final distance 4.85 m. The temporal association is evidence of an optimization disturbance alongside poorer navigation, but it does not by itself identify its cause or prove that critic overestimation was responsible. The losses subsequently return to smaller magnitudes while the success curve continues to fluctuate near one half.

The final 250 episodes have mean return -8.67 , mean length 473.82 steps, and mean final distance 3.36 m. Of these episodes, 133 succeed, 21 collide and 96 reach the cutoff. The resulting 53.2% success and 38.4% cutoff fractions explain why the long-episode trace persists. The final individual episode succeeds, but it is not representative evidence of reliable control. This particular TD3 training configuration remains much less successful than the selected DDPG and SAC configurations; the result does not establish that TD3 is generally inferior.

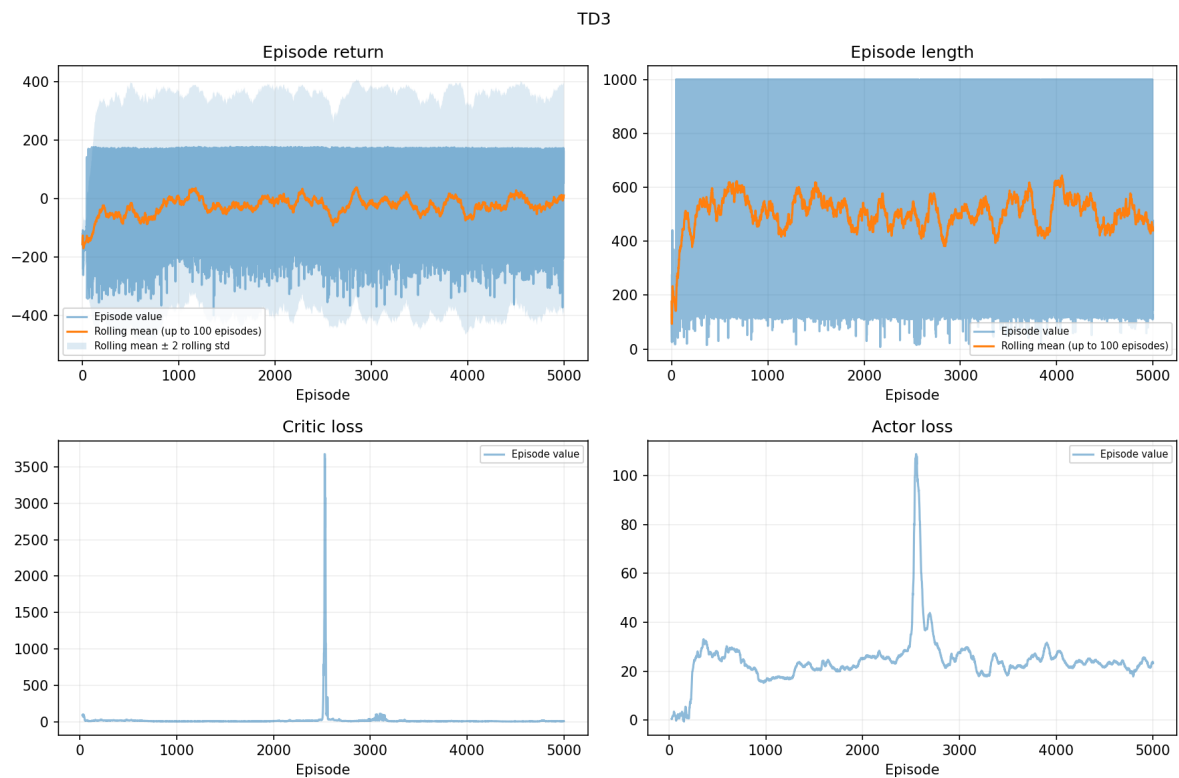


Figure 9: Selected TD3 run over 5,000 completed episodes. The reward panel includes a rolling mean $\pm$ two rolling standard deviations. A large episode-averaged critic-loss spike near episode 2,530 and a later actor-loss spike are visible. These losses are not direct measures of success or path quality.

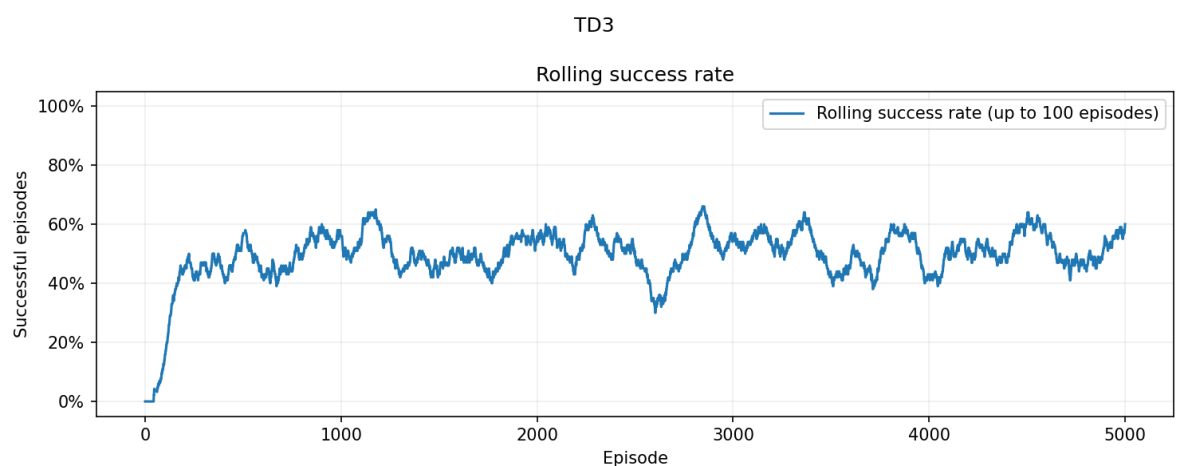


Figure 10: TD3 success rate over up to 100 recent training episodes. Following early improvement, the rate fluctuates around one half rather than approaching reliable completion. The final 100-episode value is 60%; the final 250-episode fraction is 53.2%.

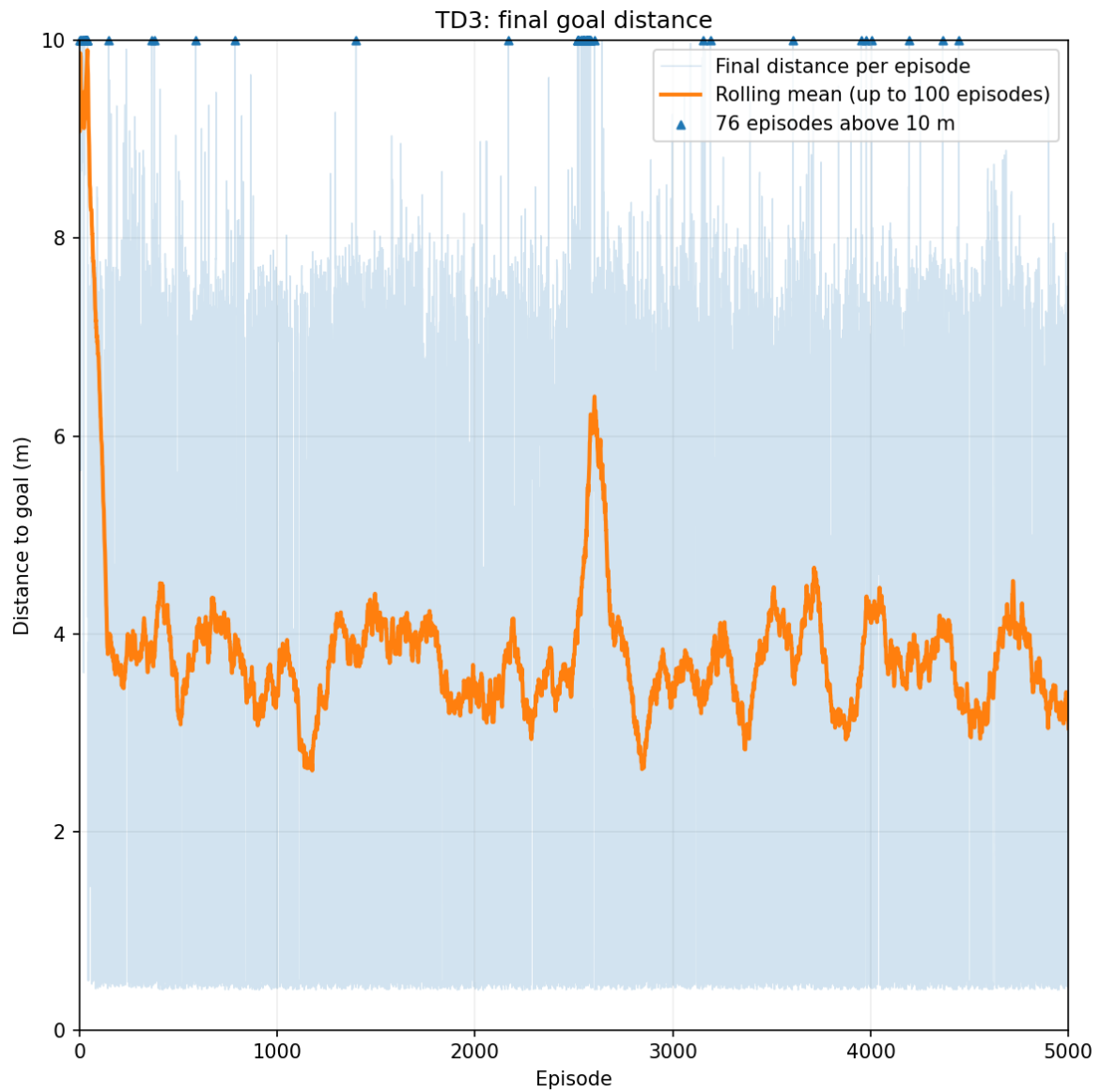


Figure 11: TD3 distance at the end of each training episode. The rolling mean stays several metres from the goal for much of the run and rises near the loss disturbance. Seventy-six episodes exceed 10 m and are marked at the upper edge; their full values remain in the mean.

3.7.3 SAC: rapid early improvement and adaptive temperature

SAC now has a complete normalized-input training record. Over episodes 1–250, mean return is -83.62 , success is 38.4%, and mean final distance is 4.40 m. By the window ending at episode 1,000, mean return reaches 138.67, success is 90.4%, mean length is 148.31 steps and mean final distance is 0.93 m. The pronounced rise in success around episodes 450–700 accompanies a reduction in both distance and episode length. This is faster early improvement on the episode axis than in the selected DDPG and TD3 runs, but different horizons, networks and update budgets prevent treating it as a controlled efficiency comparison.

Later performance remains useful but variable. The final 250 episodes yield mean return 106.71, mean length 214.08 steps and mean final distance 1.31 m. They contain 208 successes, 26 collisions and 16 cutoffs: an 83.2% success rate, below the earlier window ending at episode 1,000. The final 100-episode rate is 79%. Thus the run supports substantial learning, not monotonically improving performance or failure-free navigation.

The logged temperature starts at 0.2, drops to an episode average of approximately 0.01175 at episode 64, and subsequently rises with fluctuations, ending at approximately 0.07082. This supports the description of automatic adaptation rather than a fixed decay schedule. Critic loss rises during parts of training even while success remains relatively high; actor loss also reverses direction. Neither loss nor temperature alone supplies a navigation ranking. Older SAC evaluation outputs with a different reward scale are excluded from these new training summaries.

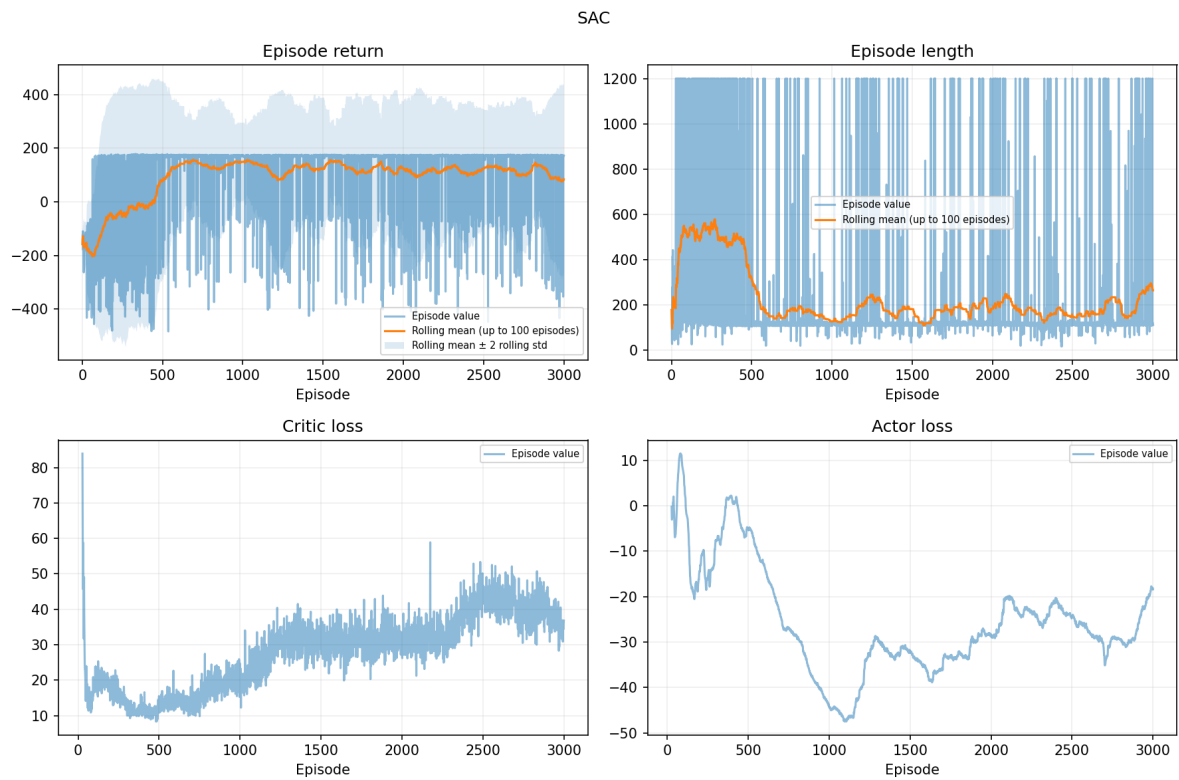


Figure 12: Selected normalized-input SAC run over 3,000 completed episodes. Return and length have rolling means over up to 100 episodes; the reward band describes two rolling standard deviations within this run. The losses are episode averages and need not decrease monotonically as the policy and targets change.

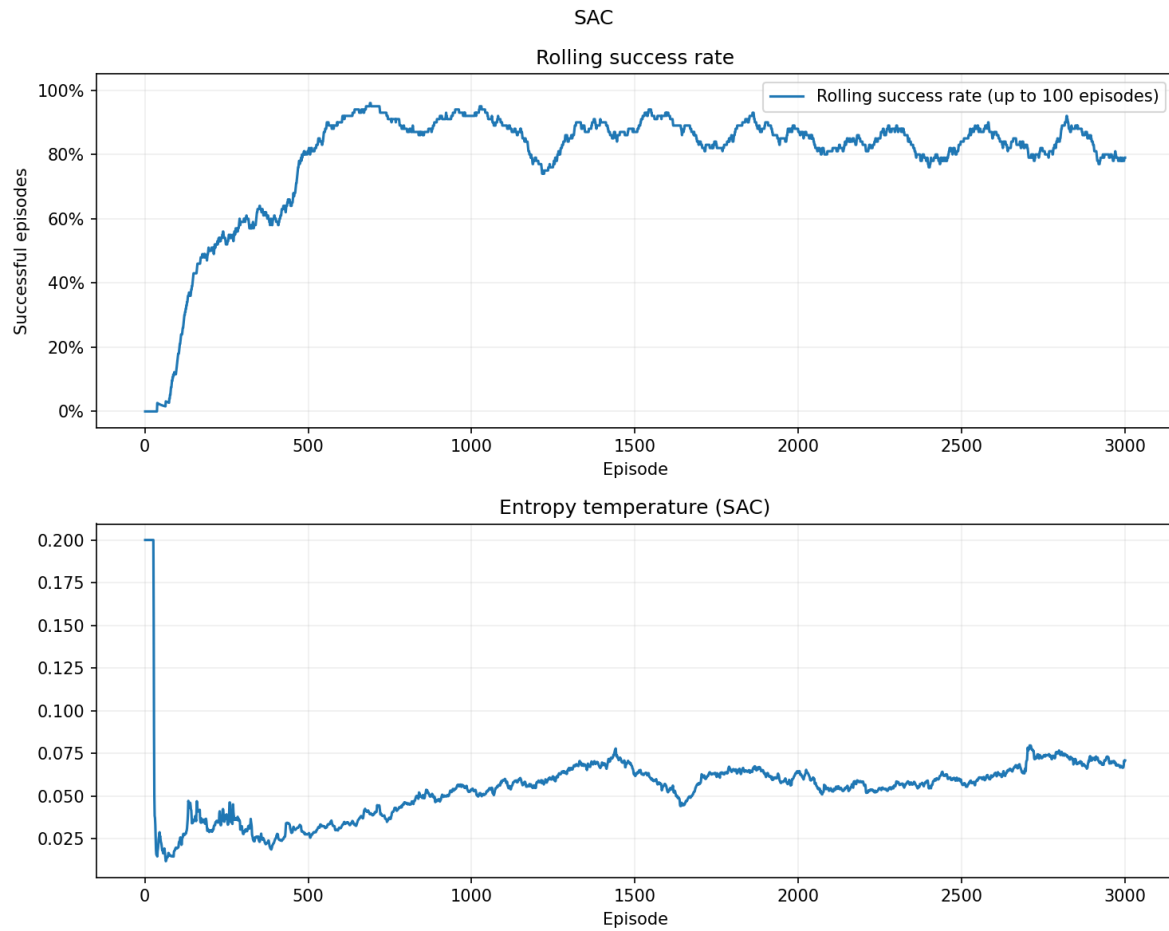


Figure 13: SAC rolling success rate and logged temperature α . The temperature trace shows episode averages over learning updates, or the current value before learning begins. It is a temperature parameter, not a direct entropy measurement. The final 100-episode success rate is 79%.

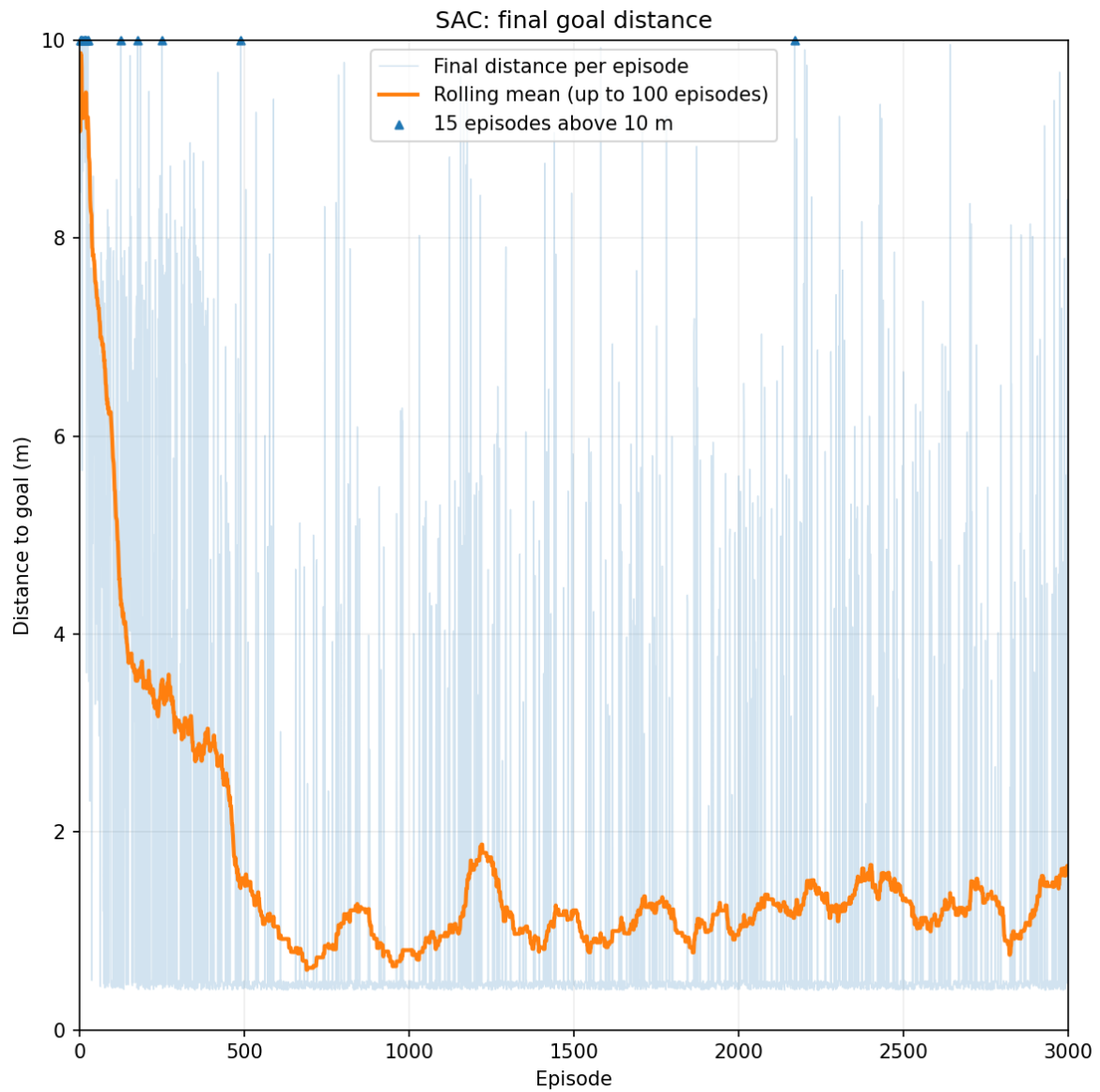


Figure 14: SAC final distance for all training episodes, with a rolling mean over up to 100 episodes and a horizontal span of 3,000 episodes. Fifteen values exceed the 10 m display limit and are marked at the upper edge. Unclipped distances determine the mean.

3.7.4 Comparison of the selected training runs

DDPG and SAC finish with substantially higher training success and smaller final distance than TD3 in these runs. At the common endpoint of 3,000 completed episodes, the preceding 250-episode mean returns are 88.17 for DDPG, -2.29 for TD3 and 106.71 for SAC, with success fractions 78.4%, 57.2% and 83.2%, respectively. Even this common episode endpoint does not equalize the numbers of environment transitions or gradient updates. The final-window table and the complete curves are descriptive evidence about these configurations; a general algorithm ranking still requires controlled evaluation and independent training seeds.

Algorithm	Last episode	Return	Success	Length	Distance (m)
DDPG	5,000	118.49	85.2%	169.04	1.29
TD3	5,000	-8.67	53.2%	473.82	3.36
SAC	3,000	106.71	83.2%	214.08	1.31

Table 9: Means over the final 250 training episodes of each selected run; success is the fraction reaching the goal. These windows end at different episode budgets and describe behavior under exploration, not evaluation of final policies or averages across independent seeds.

3.8 Evaluation evidence and historical shared-map benchmark

3.8.1 Historical three-agent benchmark

The archived comparison notebook evaluates all three agents on the same room, start, goal, and three obstacles. In $(x, y, \text{width}, \text{height})$ notation, these rectangles are

$$(2, 4, 3, 0.3), \quad (5, 2, 0.3, 3), \quad (7, 6, 1.5, 0.3).$$

The horizon is 1,000 steps. DDPG and TD3 disable exploration noise; SAC uses the transformed Gaussian mean. Table 10 transcribes the retained output of this notebook rather than a new evaluation of the local checkpoints.

All three saved policies reach the goal on this map. TD3 uses fewer steps in this particular comparison, and the return differences are small. The reward includes progress, orientation, obstacle proximity, time, and turning terms, so return is not solely a path-length measure. Repeated identical deterministic rollouts validate consistency on that map; they are not three independent training runs and do not demonstrate generalization across layouts or initial conditions.

These outputs belong to the archived comparison notebook and must not be used to rank the newly trained checkpoints shown in Section 3.7. A new three-agent comparison would require evaluating those selected checkpoints on the same scenarios and reward version. A broader ranking additionally requires diverse held-out maps and independent training replications.

3.8.2 Current DDPG evaluation

Two completed evaluation records explicitly identify the selected new DDPG checkpoint. On the fixed map above, the deterministic policy reaches the goal in 116 steps with return 168.01 and final distance 0.455 m. In a separate sample of ten mixture-generated layouts, using seeds 1234–1243, it succeeds eight times, collides once and reaches the cutoff once. Mean return is 101.56, mean length 192.7 steps and mean final distance 1.73 m over all ten episodes. The two failures are included in these averages.

The records are 20260910T181054240654Z_seed1234_93ca9597/ and 20260910T181054354166Z_seed1234_ce4ec2e7/ beneath `artifacts/continuous/ddpg/`. They support useful greedy behavior on a small sample from the implemented scenario families, not broad generalization. Corresponding current SAC and TD3 evaluations are not present in the supplied artifact folders, so this DDPG-only check is not a new three-agent ranking.

Algorithm	Return	Steps	Final distance (m)	Successes
DDPG	165.66	124	0.40	3/3
TD3	165.97	118	0.45	3/3
SAC	164.17	128	0.46	3/3

Table 10: Historical pre-fix deterministic evaluation on one shared fixed map. Each algorithm’s three episodes produce identical rounded outcomes, with no reported collisions. Returns and distances are rounded as in the notebook.

4 Discussion and limitations

4.1 Representation, exploration, and theoretical guarantees

The experiments link two different reasons for algorithm choice. In the grids, finite encodings make independent tabular values practical. In the robot environment, continuous observations and actions motivate shared neural representations and an actor to select actions without enumerating them. Parameter sharing permits generalization to unseen inputs, but it can also transfer errors between inputs; useful generalization is an empirical property, not an automatic consequence of using a network.

Cliff walking is especially informative because the difference between the two tabular targets remains visible even without neural approximation. The observed reduction in SARSA’s training falls is consistent with accounting for future exploration, whereas the continuous methods address estimation error and exploration through network and optimization choices. These are related design questions, but neither warrants transferring an observed advantage in one environment to another.

4.2 Time limits and partial observations

All implemented training workflows now treat their time limits as external cutoffs. Either termination or truncation stops data collection, but only termination removes the continuation value from a Bellman target. A fall in the cliff remains nonterminal; collecting all flags, reaching the robot goal or colliding ends the corresponding task. This explicit modeling choice differs from a finite-deadline task, which would require a remaining-time state variable and a terminal horizon.

The robot no longer adds a separate timeout penalty, so the observation’s omission of elapsed time is consistent with the external-cutoff reward definition. Sparse, finite-range LiDAR still hides parts of the map and does not uniquely identify global pose or geometry. These remaining representation limitations restrict direct use of fully observed MDP guarantees even before neural approximation. The corrected flag termination and cliff cutoff now make their recorded episode lengths consistent with their specified stopping rules.

4.3 Experimental evidence and reproducibility

The available evidence supports case studies rather than a statistically controlled ranking. The report includes one selected completed training run for each algorithm in each applicable environment, all with seed 0 and saved configuration metadata and episode logs: four tabular runs and three continuous runs. Both flag agents complete every final-block episode and achieve 27-step greedy collection. Cliff SARSA has fewer training falls and better exploratory return, while Q-learning has the shorter greedy route. DDPG and SAC show substantial improvement, while TD3 exhibits limited improvement, a large loss disturbance and persistent cutoffs in its selected run. Each grid checkpoint has one saved fixed-task greedy evaluation; DDPG also has a fixed-map evaluation and a ten-layout check.

Current source code, notebook configuration cells, old outputs and checkpoints can describe different execution histories. The training descriptions are tied to saved run metadata rather than inferred from notebook settings alone. The selected training records mark the working tree as modified; a commit identifier alone therefore cannot reconstruct every source edit used for a run. The fourteen current images—five discrete and nine continuous—are exact copies of the selected run artifacts. The accompanying figure inventory records their paths, hashes and numerical sources. Four older DDPG/TD3 images remain identifiable historical files but are not the training evidence displayed here.

Algorithm	Target and exploration	Relevant guarantee and qualification
Q-learning	Greedy maximum target; epsilon-greedy behavior.	Tabular convergence to Q^* under coverage and decreasing-step-size conditions; the finite constant-rate runs do not establish these.
SARSA	Sampled behavior-policy target; epsilon-greedy behavior.	Tabular optimal convergence under GLIE and suitable step sizes; persistent exploration changes the target being learned.
DDPG	Single critic, deterministic actor, OU exploration.	The exact deterministic policy gradient motivates the update; no general guarantee for the coupled neural implementation.
TD3	Twin target critics, delayed actor, target smoothing; OU exploration here.	Methods for reducing estimation error; no general proof of convergence or optimality for this implementation.
SAC	Stochastic actor, twin critics, entropy objective and learned temperature.	Exact soft policy iteration has an idealized guarantee; that guarantee does not transfer to approximate neural training.

Table 11: Comparison of all five algorithms in the corrected implementation. Both tabular SARSA agents carry the backup action forward.

The current training quantities come from the four discrete `metrics.csv` files identified in Section 2.1.4 and the three continuous files identified in Section 3.6; cumulative budgets come from their final rows. Discrete evaluation returns come from the saved evaluation CSVs, and route geometry was checked by replaying the selected tables without updates. The separate historical robot benchmark comes from `archive/notebooks/Continuous_Diff_Drive/compare_ddpg_sac_td3.ipynb`. A SHA-256 manifest records the preserved original notebooks. Implementation descriptions follow the active environment modules and shared `rl_project/` training infrastructure. Archived notebooks document earlier execution histories rather than supported current entry points.

The workflow preserves raw episode records, configurations, seeds and checkpoint metadata, and provides the common-scenario, multiple-training-seed experiment described above. Carrying out that full experiment remains future work. Different horizons, networks, discounts and actual update budgets constrain comparisons of the available training curves. In the grids, equal episode budgets also produce different numbers of training transitions. Success on the one fixed flag layout or cliff map does not measure generalization to new tasks. Missing current three-agent robot evaluations and uncertainty across training seeds have not been synthesized from the available single-run data.

5 Conclusions

The project implements three complementary RL tasks and five algorithms. Flag collection illustrates why the state representation must include progress toward the objectives. Both selected flag agents complete every final 250-episode training rollout and collect all five flags in 27 steps during greedy evaluation. Q-learning reaches reliable collection earlier in its recorded run, but both final greedy policies attain the minimum transition count on the specified layout.

Cliff walking demonstrates the practical difference between SARSA and Q-learning in the measured results. With ongoing exploration, SARSA’s behavior-policy backup supports a longer route with fewer falls: the final 250 training episodes contain two SARSA falls versus 102

Q-learning falls. Mean training returns are -26.91 and -62.88 respectively. Under greedy evaluation, Q-learning’s 17-step route outperforms SARSA’s 23-step detour. The benefit is therefore tied to return during exploration, rather than an unconditional advantage of SARSA.

The continuous environment extends the problem to sensor-based navigation with bounded velocity commands and a shaped reward. The selected corrected-code DDPG and SAC runs show substantial learning with remaining failures, reaching final 250-episode training success fractions of 85.2% and 83.2%. The selected TD3 run reaches 53.2%, with frequent cutoffs and a pronounced mid-training optimization disturbance. These are outcomes of particular configurations and training budgets, not a universal ranking. Current DDPG evaluation succeeds on the fixed map and eight of ten sampled layouts. The archived three-agent benchmark remains historical evidence, separate from the new training results. Broader claims require common-scenario evaluation and independent training replications.

The theoretical distinction is between exact operators and the actual sampled, approximate learning procedures. Small finite tables enable strong convergence theorems under explicit assumptions. Exact Bellman contraction can also hold in continuous spaces, but it does not guarantee convergence of the implemented neural agents. Keeping those assumptions, implementation details, and the provenance of experimental results separate produces a more accurate interpretation of both the project’s achievements and the evidence still required.

References

- [1] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018. Chapters 3, 4, 6 (including Example 6.6), and 11. <http://incompleteideas.net/book/the-book-2nd.html>.
- [2] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992. <https://www.gatsby.ucl.ac.uk/~dayan/papers/cjch.pdf>.
- [3] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning*, 2000, pp. 287–308. <https://doi.org/10.1023/A:1007678930559>.
- [4] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. *Proceedings of ICML*, PMLR 32(1):387–395, 2014. <https://proceedings.mlr.press/v32/silver14.html>.
- [5] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. arXiv:1509.02971, 2015; presented at ICLR 2016. <https://arxiv.org/abs/1509.02971>.
- [6] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. *Proceedings of ICML*, PMLR 80:1587–1596, 2018. <https://proceedings.mlr.press/v80/fujimoto18a.html>.
- [7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of ICML*, PMLR 80:1861–1870, 2018. <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- [8] T. Haarnoja et al. Soft actor-critic algorithms and applications. arXiv:1812.05905, 2018, revised 2019. <https://arxiv.org/abs/1812.05905>.